Chapter 10 — Distributed Training Systems at Scale

> *"The gap between understanding distributed training theoretically and operating a 10,000-GPU training run is the difference between knowing the rules of chess and playing a grandmaster. The pieces move the same way. The game is completely different."* — Production AI Infrastructure Principle

---

Chapter Overview

Training LLaMA-3 405B on 16,384 H100s for 54 days requires more than algorithms and hardware. It requires a systems architecture that keeps 98%+ of those GPUs computing every second, recovers from hardware failures without losing days of progress, and wastes zero cycles on data loading, communication inefficiency, or mis-configured parallelism. This chapter builds the complete mental model for designing, operating, and debugging large-scale training systems.

Sections in this chapter:

- 10.1 Training Memory Requirements — From Single GPU to Cluster
- 10.2 4D Parallelism Deep Dive — TP, PP, DP/FSDP, Context Parallel
- 10.3 ZeRO and FSDP — Memory Sharding Architecture
- 10.4 Gradient Communication — Overlap, Bucketing, and Bandwidth
- 10.5 Fault Tolerance — Checkpointing, Recovery, and Failure Modes
- 10.6 Data Pipeline Architecture — Preventing GPU Starvation
- 10.7 RLHF Infrastructure — Post-Training Alignment at Scale
- 10.8 Pre-Launch Checklist and Eight Numbers
- 10.9 Training Run War Stories — What Failures Actually Look Like [r1]
- 10.10 Resuming and Recovery — Checkpoint Validation [r1]
- 10.11 Key Takeaways and Review Questions

10.1 Training Memory Requirements — From Single GPU to Cluster

10.1.1 The 16-Bytes-Per-Parameter Rule

Every training system bottoms out at this fundamental accounting:

```
ADAM OPTIMIZER MEMORY BREAKDOWN (mixed-precision training):

  Component                  Dtype   Bytes/param   Notes
  ──────────────────────────────────────────────────────────────────────
  BF16 model weights         BF16    2             Forward pass compute
  FP32 master weights        FP32    4             Optimizer reference copy
  FP32 gradients             FP32    4             Accumulated during backward
  Adam first moment (m)      FP32    4             EMA of gradients
  Adam second moment (v)     FP32    4             EMA of gradient squares
  ──────────────────────────────────────────────────────────────────────
  TOTAL                              18 bytes/param

  Simplification used in practice: ~16 bytes/param (rounds down BF16 grads)

  LLaMA-3 70B TRAINING STATE:
    Parameters:     70 × 10⁹
    Total memory:   70B × 16B = 1,120 GB = 1.12 TB minimum
    H100 capacity:  80 GB per GPU
    GPUs needed:    1,120 / 80 = 14 GPUs (raw state, no activations)
    With activations + safety margin: 20-32 GPUs minimum

  LLaMA-3 405B TRAINING STATE:
    405B × 16B = 6.48 TB minimum
    H100 GPUs:  6,480 / 80 = 81 GPUs (absolute minimum; real deployments: 256+)
```

10.1.2 Activation Memory — The Variable Term

```
ACTIVATION MEMORY (one transformer block, BF16):

  Per-layer activation storage (forward pass, needed for backward):
    Input to block:         S × d_model × 2B
    Attention Q,K,V:        3 × S × d_model × 2B
    Attention scores:       H × S × S × 2B  (S² term — eliminated by FlashAttn!)
    MLP intermediate:       S × d_ff × 2B
    Total per layer:        ~4 × S × d_model × 2B  (FlashAttention, no S² term)

  For LLaMA-3 70B (S=4096, d=8192, d_ff=28672, BF16, no checkpointing):
    Per layer: 4 × 4096 × 8192 × 2 = 268 MB
    80 layers: 80 × 268 MB = 21.4 GB activation storage per micro-batch!

  WITH SELECTIVE ACTIVATION CHECKPOINTING (store only layer inputs):
    Store: 1 × S × d_model × 2B per layer = 67 MB per layer
```

```
    80 layers: 80 × 67 = 5.4 GB (4× reduction in activation memory)
    Recompute cost: ~30% extra FLOPs (forward pass rerun during backward)

  WITH MIN-CUT RECOMPUTATION (Chapter 9, torch.compile):
    ~5.4 GB activation memory at ~5% extra FLOPs (not 30%)
    The optimal choice when torch.compile is active.
```

10.2 4D Parallelism Deep Dive

Modern large-model training uses four orthogonal parallelism strategies simultaneously. Choosing the right configuration is the highest-leverage architectural decision in training system design.

10.2.1 Tensor Parallelism — Within-Layer Weight Splitting

```
TENSOR PARALLELISM (Megatron-style):

  Splits individual weight matrices across GPUs.
  Every GPU holds a column-wise or row-wise SHARD of every weight.

  COLUMN PARALLEL (QKV projections, FFN gate/up):
    W [d_model × d_ff] split into TP shards:
    GPU 0: W[:, 0 : d_ff/TP],  GPU 1: W[:, d_ff/TP : 2d_ff/TP], ...
    Forward: Y_i = X × W_i   (X broadcast, each GPU computes partial output)
    No communication in forward — each GPU has its output shard.

  ROW PARALLEL (output projections, FFN down):
    W [d_ff × d_model] split into TP shards:
    GPU 0: W[0:d_ff/TP, :],  GPU 1: W[d_ff/TP:2d_ff/TP, :], ...
    Forward: Z_i = Y_i × W_i  (Y_i is the output of column-parallel above)
    AllReduce to sum partial: Z = Z_0 + Z_1 + ... + Z_TP-1

  COMMUNICATION PER LAYER:
    1 AllReduce per attention sub-layer + 1 per FFN sub-layer = 2 per block
    Message size: batch × seq_len × d_model × bytes_per_element

  EXAMPLE — LLaMA-3 70B, TP=8, micro-batch=4, S=4096:
    Message size = 4 × 4096 × 8192 × 2B = 268 MB per AllReduce
    Over NVLink (600 GB/s bidirectional):
      Ring AllReduce = 2×(TP-1)/TP × 268 MB / (600/TP) GB/s
                     = 2×7/8 × 268 MB / 75 GB/s ≈ 6.2 ms per AllReduce
      Wait — simplified: 268 MB / 600 GB/s = 0.45 ms (all-bandwidth view)
```

```
      80 layers × 2 AllReduces: 80 × 2 × 0.45 = 72 ms per step ← acceptable
    Over InfiniBand NDR (50 GB/s per rail):
      268 MB / 50 GB/s = 5.4 ms per AllReduce
      80 × 2 × 5.4 = 864 ms per step ← FATAL (larger than compute time!)

  THE TP GOLDEN RULE (covered Ch 3A, repeated here for training context):
    TP MUST STAY WITHIN THE NVLINK DOMAIN.
    TP degree ≤ GPUs within one DGX/HGX node (8 on standard DGX H100).
    Never run TP across InfiniBand — 12× latency penalty destroys MFU.
```

10.2.2 Sequence Parallelism — The TP Complement

```
SEQUENCE PARALLELISM (Megatron-LM 2023):

  Problem: With TP, LayerNorm and Dropout run on FULL replicated activations.
  Each TP GPU holds identical [S × d_model] tensors for non-GEMM ops.
  Memory waste: TP GPUs each store complete copies of non-sharded activations.

  Solution: Shard the sequence dimension during non-GEMM operations.
    Before column-parallel GEMM: AllGather sequence shards → full sequence
    After row-parallel GEMM:     ReduceScatter → shard sequence again
    LayerNorm, Dropout:          Computed on [S/TP × d_model] shards locally

  MEMORY SAVINGS (LLaMA-3 70B, TP=8, BF16):
    Without SP: 80 layers × 268 MB (full activation) = 21.4 GB per micro-batch
    With SP:    80 layers × 268/8 MB = 2.68 GB per micro-batch
    Savings: 18.7 GB per micro-batch — critical for large batch training

  RULE: Always enable SP when TP > 1.
    Megatron-LM: --sequence-parallel (default in recent versions)
    Cost: additional AllGather + ReduceScatter per block (same bandwidth as
existing)
    Net: zero extra communication, significant memory reduction.
```

10.2.3 Pipeline Parallelism — Cross-Node Layer Splitting

```
PIPELINE PARALLELISM (PP):

  Assigns contiguous layer groups to different GPU stages.
  Activations flow forward stage → stage; gradients flow backward.
  Spans nodes over InfiniBand (one hop between adjacent stages).

  NAIVE (GPipe) — AVOID:
```

```
    Stage 0: runs all M microbatches forward → waits
    Stage 1: starts only after Stage 0 finishes all forward passes
    GPU utilization: terrible for small M; pipeline is mostly idle.

  1F1B SCHEDULE (One Forward, One Backward — the production standard):
    Each stage alternates forward and backward for different microbatches.
    Pipeline fill/drain phases create "bubbles" (idle time).

  BUBBLE FRACTION FORMULA:
    bubble = (PP - 1) / (PP - 1 + M)

    Where PP = pipeline stages, M = microbatches per step

    PP=4, M=8:   bubble = 3/11 = 27%  (too high)
    PP=4, M=32:  bubble = 3/35 = 8.6%  (acceptable)
    PP=4, M=128: bubble = 3/131 = 2.3%  (good)
    PP=8, M=32:  bubble = 7/39 = 17.9%  (borderline)

    RULE: Set M ≥ 4 × PP to keep bubble below 10%.

  INTERLEAVED 1F1B (Megatron-LM 2021):
    Assigns multiple non-contiguous model chunks per GPU stage.
    Each GPU holds V chunks (interleaved across the model).
    Bubble fraction: (PP - 1) / (PP - 1 + M × V)
    At V=2: same bubble fraction but requires more micro-batches in flight.
    Memory cost: V × activation_per_stage (more in-flight activations).
    Use when: bubble > 10% AND memory allows V=2 without OOM.

  PP COMMUNICATION:
    Point-to-point activation sends between adjacent stages.
    Volume per microbatch: micro_B × S × d_model × 2B
    LLaMA-3 70B, micro_B=1, S=4096: 1 × 4096 × 8192 × 2 = 67 MB per P2P send
    With PP=4, M=32: 3 × 32 × 67 MB = 6.4 GB per step
    Compared to DP AllReduce (140 GB): PP activation traffic is small.
```

10.2.4 Data Parallelism and ZeRO — Across Replicas

```
DATA PARALLELISM (DP):

  Each DP replica holds a complete model copy + optimizer state.
  Each replica processes different input batches (data shards).
  After backward: AllReduce gradients across all replicas to synchronize.
```

```
  COMMUNICATION:
    AllReduce volume = N_params × 2B (BF16 gradients)
    LLaMA-3 70B: 70B × 2B = 140 GB per AllReduce step
    Ring AllReduce with DP=16: each GPU sends/receives 2 × (15/16) × 140 GB = 262
GB
    Over InfiniBand NDR (50 GB/s per link): 262 GB / 50 GB/s = 5.24 seconds!

  WITHOUT ZERO: DP AllReduce is 5+ seconds per step → catastrophic.
  WITH ZERO (next section): AllReduce replaced by ReduceScatter + AllGather,
  overlapped with backward pass → hidden behind compute.

CONTEXT PARALLELISM (CP) — LONG SEQUENCES:
  Splits the sequence dimension across GPUs for attention.
  Each GPU computes attention for [S/CP] query tokens attending to ALL keys.
  Communication: AllGather K, V tensors across CP group before attention.

  Use case: S > 32K tokens (long context training).
  Memory: activation memory ∝ S/CP per GPU (linear scaling).
  At S=128K, CP=16: effective per-GPU sequence = 8K — manageable!

  Meta's LLaMA-3 training: 4D parallelism = TP × PP × CP × DP simultaneously.
```

10.2.5 4D Parallelism Configuration Examples

```
PARALLELISM SELECTION GUIDE — BY MODEL AND CLUSTER SIZE:

  EXAMPLE 1: LLaMA-3 70B on 512× H100 (64 nodes × 8 GPUs):

  Memory check:
    70B × 16B = 1,120 GB total training state
    With TP=8 + ZeRO-2: ~1,120/8/8 = 17.5 GB per GPU + activations

  Configuration: TP=8, PP=4, DP=16
    TP=8: within each 8-GPU node (NVLink)
    PP=4: across 4 consecutive nodes (IB point-to-point)
    DP=16: 512 / (8 × 4) = 16 data-parallel replicas
    Microbatches: M = 32 → bubble = 3/35 = 8.6% ✓

  Global batch size: DP × M × micro_batch_size = 16 × 32 × 4 = 2,048 sequences

  EXAMPLE 2: LLaMA-3 405B on 4,096× H100 (512 nodes):

  Memory check:
```

```
    405B × 16B = 6,480 GB total
    With TP=8 + PP=8: 6,480 / 64 = 101 GB per TP-PP group → too tight
    Add ZeRO-1 (optimizer shard within DP): saves ~3/4 optimizer state
    Effective: ~50 GB per GPU → fits with headroom ✓

  Configuration: TP=8, PP=8, DP=64
    4,096 / (8 × 8) = 64 data-parallel replicas
    M = 64 → bubble = 7/71 = 9.9% ← borderline; consider interleaved V=2
    Global batch: 64 × 64 × 2 = 8,192 sequences

  MNEMONIC FOR CONFIGURATION ORDER:
    1. TP=8 (always start here — fills NVLink domain)
    2. PP = ceiling(model_size / (TP × GPU_HBM × 0.7))  (fill remaining nodes)
    3. DP = total_GPUs / (TP × PP)  (whatever remains)
    4. Add CP only if S > 32K tokens
```

10.3 ZeRO and FSDP — Memory Sharding Architecture

10.3.1 ZeRO Stages — Progressive Memory Sharding

```
ZERO OPTIMIZER STAGES (Microsoft DeepSpeed):

  Standard DP baseline (no ZeRO):
    Every GPU stores: weights + gradients + optimizer states (m, v, master
weights)
    Per GPU: 16 bytes/param × N_params

  ZERO STAGE 1 (optimizer state partitioning):
    What's sharded: optimizer states (m, v, master weights) — 12 bytes/param
    Each GPU stores: optimizer state for its N/DP parameter shard
    Communication: ReduceScatter for gradients + AllGather for params after update
    Per GPU: (16 - 12×(1-1/DP)) bytes/param ≈ 4 + 12/DP bytes/param
    For DP=16: 4 + 0.75 = 4.75 bytes/param vs 16 bytes (3.4× reduction)

  ZERO STAGE 2 (+ gradient partitioning):
    What's sharded: gradients + optimizer states — 14 bytes/param
    Per GPU: 2 + 14/DP bytes/param
    For DP=16: 2 + 0.875 = 2.875 bytes/param (5.6× reduction)

  ZERO STAGE 3 (+ parameter partitioning — full sharding):
    What's sharded: everything — 16 bytes/param
```

```
    Per GPU: 16/DP bytes/param
    For DP=16: 1 byte/param (16× reduction!)
    Communication: AllGather parameters in forward, ReduceScatter gradients in
backward
    Total bandwidth: 3× the AllReduce equivalent (vs 1× for stage 1)

  ZERO STAGE COMPARISON:

    Stage  Memory/GPU     Communication    Best For
    ──────────────────────────────────────────────────────────────────
    0 (DP) 16 bytes/param 1× AllReduce     Models < 7B (fit with standard DP)
    1      ~4.75/param    1.5× AllReduce   7B-70B, DP ≥ 16
    2      ~2.875/param   1.5× AllReduce   70B-200B, when mem-constrained
    3      16/DP /param   3× AllReduce     Very large models; accept BW cost
    ──────────────────────────────────────────────────────────────────


  THE COMMUNICATION TRADE-OFF:
    ZeRO Stage 3: 3× more bytes transferred vs naive AllReduce.
    How can this be better?

    Stage 3 ReduceScatter + AllGather CAN BE OVERLAPPED with compute.
    Naive AllReduce: blocking synchronization — GPU waits.
    Stage 3 with prefetch: GPU computes next layer while current layer's
    parameters are being fetched via AllGather.
    Net effect: 3× more data, but most hidden behind compute.
    MFU impact: ~5-15% overhead (vs 25-50% for non-overlapped AllReduce).
```

10.3.2 FSDP — PyTorch Native ZeRO Stage 3

```python
```

```python
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP,
    MixedPrecision,
    BackwardPrefetch,
    ShardingStrategy,
)
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy

# FSDP CONFIGURATION (production LLM training):
policy = transformer_auto_wrap_policy(
    transformer_layer_cls={TransformerBlock}  # wrap each transformer block
)
model = FSDP(
    model,
    auto_wrap_policy=policy,
    sharding_strategy=ShardingStrategy.FULL_SHARD,    # ZeRO Stage 3
    mixed_precision=MixedPrecision(
        param_dtype=torch.bfloat16,     # compute in BF16
        reduce_dtype=torch.float32,     # accumulate gradients in FP32
        buffer_dtype=torch.bfloat16,    # buffers in BF16
    ),
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,  # overlap comm+compute
    forward_prefetch=True,                             # prefetch next layer's param
    limit_all_gathers=True,                            # memory safety
    use_orig_params=True,                              # required for torch.compile
)

# WHEN TO USE EACH SHARDING STRATEGY:
#   FULL_SHARD     (ZeRO Stage 3): max memory savings, 3× comm
#   SHARD_GRAD_OP  (ZeRO Stage 2): gradient + optimizer sharding only
#   NO_SHARD       (DDP):          standard data parallel, no sharding
#   HYBRID_SHARD:                  full shard within node, replicate across nodes
#     Best for large clusters where cross-node AllGather is expensive
```

10.3.3 DeepSpeed ZeRO vs FSDP

```
DEEPSPEED vs FSDP COMPARISON:

                      DeepSpeed ZeRO          PyTorch FSDP
  ──────────────────────────────────────────────────────────────
  ZeRO Stage 3        Yes                     Yes (FULL_SHARD)
  ZeRO-Infinity       Yes (NVMe offload)       No
```

```
  CPU offload         Yes (params + optim)     Limited (grad offload only)
  Gradient compress   Yes (1-bit Adam, etc)    No
  PyTorch integration External library         Native (PyTorch 1.12+)
  torch.compile compat Limited                 Yes (use_orig_params=True)
  Hybrid sharding     Via ZeRO++               Yes (HYBRID_SHARD)
  Ease of use         Harder config            Simpler API

  USE DEEPSPEED WHEN:
    Models > 1T parameters requiring CPU/NVMe offload
    Gradient compression needed (slow IB links < 200G)
    Fine-tuning large models on single-node with tight memory

  USE FSDP WHEN:
    Standard large-model training on NVLink+IB clusters
    Composing with TP/PP (via Megatron+FSDP via NeMo or FSDP+TP)
    PyTorch-native toolchain preference
    torch.compile integration needed

  PLACEMENT RULE:
    TP:   within NVLink domain (8 GPUs, same node)
    FSDP: spans across nodes over InfiniBand
    PP:   one IB hop between adjacent pipeline stages

    MNEMONIC: "Small frequent comms → NVLink; Large infrequent → IB"
    TP AllReduce (268 MB, 160× per step): NVLink
    FSDP AllGather (140 GB, 1× per step): InfiniBand
```

10.4 Gradient Communication — Overlap, Bucketing, and Bandwidth

10.4.1 Gradient Bucketing — Why and How

```
THE PER-PARAMETER ALLREDUCE PROBLEM:
  A 70B model has ~70,000 individual parameter tensors.
  Each tensor may be tiny: bias vector = [8,192] × 4 bytes = 32 KB.

  Launching one AllReduce per tensor:
    70,000 × NCCL launch overhead (~30-100 μs) = 2-7 seconds overhead!
    Even if each AllReduce is instant: the overhead alone exceeds step time.

  BUCKET-BASED ALLREDUCE (DDP / FSDP default):
    Accumulate gradients into buckets (default: 25 MB per bucket)
```

```
    Launch AllReduce when bucket fills (one NCCL call per ~2 MB of params)
    For 70B: 140 GB / 25 MB ≈ 5,600 NCCL calls (vs 70,000 per-param calls)
    Overhead reduction: 12.5× fewer launches

BUCKET SIZE TUNING:
  Small buckets (< 5 MB):   More NCCL calls but earlier overlap (better for fast
nets)
  Large buckets (> 50 MB):  Fewer NCCL calls but delayed gradient sync

  For 70B on 512× H100 with IB NDR:
    Optimal: ~25-50 MB buckets
    Set via: DDP/FSDP bucket_cap_mb parameter

  PyTorch DDP:
    model = DDP(model, bucket_cap_mb=25)

  Megatron-LM:
    --overlap-grad-reduce   (overlap AllReduce with backward)
    --bucket-size 50        (50 MB buckets)
```

10.4.2 Communication-Computation Overlap

```
OVERLAP STRATEGY — HIDING ALLREDUCE BEHIND BACKWARD PASS:

WITHOUT OVERLAP:
  Forward pass:     100% compute
  Backward pass:    100% compute
  AllReduce:        100% communication (GPU waits!)
  Step time = forward + backward + AllReduce
  If AllReduce = 50% of backward: 50% of backward time is wasted waiting.

WITH OVERLAP (DDP default behavior):
  Forward pass: compute
  Backward pass: compute layer L → AllReduce(gradients of layer L) launches
                 → while layer L-1 backward computes, layer L AllReduce runs
                 → overlap compute and communication

  Key requirement: enough gradient buffers to keep both NCCL and GPU active.
  Achieved via: backward hook triggers AllReduce bucket launch.

OVERLAP VERIFICATION:
  1. Nsight Systems trace: NCCL operations should appear BELOW backward compute
     kernels in the timeline (running concurrently on separate CUDA streams).
```

```
  2. Total step time should be ≈ max(backward_time, allreduce_time), not sum.
  3. If NCCL appears AFTER backward in trace: overlap broken.

  COMMON OVERLAP FAILURES:
  a. Gradient accumulation without sync:
     DDP fires AllReduce after EVERY micro-batch.
     With gradient accumulation (M steps): need to prevent early AllReduce.
     Fix: use model.no_sync() context for accumulation steps.

  b. Custom backward hooks blocking NCCL:
     Any Python code in the backward hook delays NCCL launch.
     Fix: keep hooks minimal; pre-compute outside the hook.

  c. Gradient norm clipping before AllReduce:
     Norm requires all gradients → waits for full backward → then AllReduces.
     Fix: compute global norm via AllReduce before clipping, or clip per-replica.

ALLREDUCE BANDWIDTH MATH (LLaMA-3 70B, DP=16, ring AllReduce):
  Volume per GPU = 2 × (DP-1)/DP × gradient_size
                 = 2 × 15/16 × 140 GB = 262.5 GB per step per GPU

  With 1× IB NDR link (50 GB/s):  262.5 GB / 50 GB/s = 5.25 seconds  ← must
overlap!
  With 8× IB NDR links:           262.5 GB / 400 GB/s = 0.66 seconds  ←
overlappable

  This is why each GPU in a production training cluster needs 8 IB HCA ports
  (rail-optimized topology: 1 IB link per GPU, all connected to the same switch
rail).
```

10.5 Fault Tolerance — Checkpointing, Recovery, and Failure Modes

10.5.1 Failure Rates at Scale

```
GPU FAILURE PROBABILITY AT TRAINING SCALE:

  Rough empirical baseline: ~1% per GPU per week hardware failure rate
  (includes HBM errors, NVLink failures, PCIe issues, power anomalies)

  Cluster size   Expected failures/week   Job interruption frequency
  ──────────────────────────────────────────────────────────────────
```

```
128 GPUs       ~1.3/week                ~every 5 days
1,024 GPUs     ~10.2/week               ~every 17 hours
10,000 GPUs    ~100/week                ~every 1.7 hours
16,384 GPUs    ~164/week                ~every 1 hour
──────────────────────────────────────────────────────────────────────

At 10K GPUs: 100 failures/week = hardware failure every ~1.7 hours.
At 30-day training run: ~420 expected failures total.

Without fault tolerance: the run restarts from zero every 1.7 hours.
With async checkpointing: the run loses at most ~5 minutes of progress.

This is why fault tolerance is not a feature — it is the prerequisite for
any training run at scale exceeding 1,000 GPUs.
```

10.5.2 Checkpointing Architecture

```
SYNCHRONOUS vs ASYNCHRONOUS CHECKPOINTING:

  SYNCHRONOUS (naive):
    All GPUs stop training.
    Write full checkpoint to storage (model weights + optimizer + metadata).
    Resume training.

    Write time for 70B BF16:
      Checkpoint size ≈ model (140 GB) + optimizer (840 GB) = 980 GB
      Storage bandwidth (Lustre, 512 GPUs): ~100 GB/s aggregate
      Write time: 980 GB / 100 GB/s = 9.8 seconds
      At step time 2 seconds: 9.8/2 = 4.9 steps of wasted time
      Every 100 steps: 4.9/100 = ~5% overhead — acceptable but avoidable.

    Write time for 405B:
      Checkpoint ~5.5 TB; 54 seconds at same bandwidth.
      Every 100 steps: 54/200 = 27% overhead — unacceptable.

  ASYNCHRONOUS CHECKPOINTING:
    Step N completes: optimizer state snapshot pinned to CPU RAM.
    Step N+1 begins immediately on GPU.
    Background thread writes CPU RAM snapshot to storage.

    Requirement: enough CPU RAM to hold one checkpoint snapshot.
    For 70B: ~980 GB checkpoint = 980 GB CPU RAM needed per node.
    DGX H100: 2 TB CPU DRAM — 1 checkpoint fits comfortably.
```

```
    Overhead: <0.5% per step (snapshot pinning is fast; write is asynchronous)

    Framework support:
      Megatron-LM: --async-save  (enabled by default in recent versions)
      PyTorch:     torch.save with async=True (via multiprocessing)
      DeepSpeed:   asynchronous_checkpoint=true in config

OPTIMAL CHECKPOINT INTERVAL:
  Checkpointing too frequently: wasted IO bandwidth.
  Checkpointing too rarely: large amount of recompute after failure.

  OPTIMAL INTERVAL (Daly's formula for restart-from-checkpoint):
    T_opt = sqrt(2 × T_checkpoint × T_MTTR)

    Where:
      T_checkpoint = time to write one checkpoint
      T_MTTR = mean time to recover after failure (includes detection + restart)

    Example (70B, T_ckpt=10s, T_MTTR=600s):
      T_opt = sqrt(2 × 10 × 600) = sqrt(12,000) = 109 seconds
      At step time 2s: checkpoint every ~55 steps

    Example (405B, T_ckpt=54s, T_MTTR=1200s):
      T_opt = sqrt(2 × 54 × 1200) = sqrt(129,600) = 360 seconds
      At step time 4s: checkpoint every ~90 steps

    RULE OF THUMB: Checkpoint every sqrt(2 × recovery_overhead) steps.
    For very expensive recovery: checkpoint more frequently.
```

10.5.3 Failure Mode Taxonomy

```
TRAINING FAILURE TAXONOMY — BY DETECTION AND RECOVERY:

  CATEGORY A: HARDWARE FAILURES (easy to detect, handled by framework)
    GPU HBM ECC uncorrectable error → CUDA context crash → job abort → auto-
restart
    NVLink failure → collective hangs → watchdog timeout → job abort → auto-
restart
    CPU/PCIe fault → node dies → SLURM detects → node replaced → job restarts
    Detection: immediate (CUDA error) or 30-60 seconds (NCCL watchdog timeout)
    Recovery: restart from last checkpoint; replace failed node with hot-standby
```

```
  CATEGORY B: SILENT HARDWARE DEGRADATION (hard to detect)
    Degraded IB link (400G → 100G): AllReduce slower but not failed
    GPU memory throttle (HBM heating): compute slower but no crash
    NVLink soft errors (increased retry rate): latency spikes but no abort
    Detection: custom monitoring required (ibstatus, DCGM link speed)
    Recovery: identify and drain the affected node; replace with hot-standby

  CATEGORY C: TRAINING INSTABILITY (hardest to detect)
    Loss spike / divergence: gradient explodes → loss goes to NaN/inf
    Gradient clipping masks real problems: norm clip hides divergence
    Data corruption: bad batch produces silently wrong gradients
    Silent weight corruption: ECC correctable errors accumulate
    Detection: monitor loss curve + gradient norm + per-step variance
    Recovery: roll back to last known-good checkpoint; analyze root cause

NCCL WATCHDOG TIMEOUT:
  NCCL provides a timeout for collective operations.
  If an AllReduce doesn't complete in N seconds: all ranks abort.
  Set via: NCCL_TIMEOUT_SECONDS=1800  (30 minutes for large clusters)

  Too short: false positives from slow nodes or long checkpoints.
  Too long: a failed node hangs the entire cluster for minutes.
  Recommended: 1800s (30 min) for clusters > 1K GPUs.
```

10.6 Data Pipeline Architecture

10.6.1 The GPU Starvation Problem

```
DATA PIPELINE BOTTLENECK PATTERN:

  GPU training step: 1-2 seconds  (H100, large batch)
  Data loading (naive, single thread): 0.5-5 seconds per batch

  If data loading > GPU step time:
    GPUs wait for data → MFU collapses to < 50% despite perfect GPU code.

  SYMPTOMS:
    nsys profile: gaps between consecutive forward pass kernels
    CPU utilization: one core at 100%, GPUs idle
    SM active %: low between steps (not between kernels — that's compute-bound)
```

```
DATA PIPELINE REQUIREMENTS:
  Data loading throughput must exceed GPU consumption rate by 2-3× margin.
  Target: data pipeline runs entirely in background; GPU NEVER waits for data.
```

10.6.2 Production Data Pipeline Design

```
PRODUCTION DATA PIPELINE (LLaMA-3 style):

  OFFLINE PREPROCESSING (done once, before training):
    1. Download raw text corpus → pre-tokenize with the model's tokenizer.
    2. Serialize to large binary shards (>100M tokens / file).
       Eliminates repeated tokenization overhead at train time.
       Reduces file count (fewer IOPS against distributed FS).
    3. Shuffle at shard level (Fisher-Yates over shard list).
    4. Write to high-throughput distributed filesystem (Lustre, GPFS, or
equivalent).

  ONLINE (at training time):
    1. Memory-mapped files (np.memmap) for O(1) random access within shards.
    2. DataLoader with num_workers ≥ 4 (async prefetch from memory-mapped files).
    3. pin_memory=True in DataLoader → faster Host→GPU CUDA transfer.
    4. Prefetch queue: maintain N=4+ batches ahead of GPU consumption.
    5. Sequence packing (next section) to eliminate padding waste.

  PREFETCH OVERLAP:
    DataLoader on CPU prefetches batch K+1 while GPU trains on batch K.
    Requires: CUDA non-blocking transfers + separate CUDA stream for copy.

    torch implementation:
      for batch in dataloader:
          # Non-blocking transfer to GPU while previous GPU step runs
          inputs = batch['input_ids'].cuda(non_blocking=True)
          # ... training step

SEQUENCE PACKING — 20% FREE THROUGHPUT:

  PROBLEM — PADDING WASTE:
    Typical training data has variable-length documents.
    Without packing: each sequence padded to seq_len=4096.
    Example batch: [256 tokens, 512, 128, 4096] → padded to [4096, 4096, 4096,
4096]
    Wasted compute: (4096-256 + 4096-512 + 4096-128) / (4 × 4096) = 85% wasted!
```

```
  SOLUTION — SEQUENCE PACKING:
    Pack multiple documents into each sequence up to seq_len.
    Use attention mask to prevent attention across document boundaries.

    Example (seq_len=4096):
      Sequence 1: [doc_A=2048 tokens][doc_B=1024][doc_C=1024]
      Sequence 2: [doc_D=512][doc_E=3584]
    All sequences now 4096 tokens → zero padding waste.

    Attention masking: block-diagonal within document boundaries.
    Token in doc_B cannot attend to tokens in doc_A (different documents).
    FlashAttention varlen mode: flash_attn_varlen_qkvpacked_func.

  THROUGHPUT IMPACT:
    Average packing efficiency on typical web corpus: 95-98%.
    Throughput gain vs unpackaged: 15-25%.
    For 70B on 512× H100: ~20% gain = effectively 102 free GPU-equivalents.
```

10.7 RLHF Infrastructure

10.7.1 The RLHF System Architecture

```
RLHF/PPO SYSTEM ARCHITECTURE (4 concurrent processes):

┌──────────────────────────────────────────────────────────────────┐
│  Process 1: ROLLOUT WORKERS (inference workload)                 │
│  Prompts → Policy Model → Generate responses                     │
│  Requires: continuous batching, KV cache, fast decoding          │
│  GPU budget: 30-40% of allocation                                │
└──────────────────────────┬──────────────────────────────────────┘
                           │ (prompt, response) pairs
┌──────────────────────────▼──────────────────────────────────────┐
│  Process 2: REWARD SCORER (inference, smaller model)             │
│  (Prompt, Response) → Reward Model → Scalar reward [0,1]         │
│  Must match rollout throughput rate                              │
│  GPU budget: 10-20% (reward model typically 7-13B)               │
└──────────────────────────┬──────────────────────────────────────┘
                           │ reward signals
┌──────────────────────────▼──────────────────────────────────────┐
│  Process 3: REFERENCE MODEL (frozen SFT model, KL check)         │
│  Response → Reference Model → log probabilities                  │
```

```
│  KL penalty = KL(policy || reference)                          │
│  Memory: full model copy in memory (can share with policy)    │
└────────────────────────┬──────────────────────────────────────┘
                         │ KL penalty
┌────────────────────────▼──────────────────────────────────────┐
│  Process 4: PPO POLICY UPDATE (training workload)             │
│  (response, reward, KL) → PPO clip loss → policy update       │
│  Standard backward pass + AllReduce                           │
│  GPU budget: 40-50% of allocation                             │
└───────────────────────────────────────────────────────────────┘

BOTTLENECK: Rollout generation (inference) is 3-5× slower per token than training.
Result: GPU utilization is unbalanced — training GPUs often idle.
Solution: separate GPU pools with async queuing between processes.
```

10.7.2 PPO Memory Overhead

```
PPO MEMORY REQUIREMENTS (LLaMA-3 70B policy, 13B reward, 70B reference):

  Policy model (full training state):    70B × 16B = 1,120 GB
  Reference model (weights only):        70B × 2B  =   140 GB  (frozen, no optim)
  Reward model (weights only, 13B):      13B × 2B  =    26 GB
  KV cache (rollout inference):          ~50 GB (batch=32, S=2048)
  ──────────────────────────────────────────────────────────────────────
  Total:                                                  ~1,336 GB

  On 128× H100 (10,240 GB):
    Policy (ZeRO-3, DP=16): 1,120/16 = 70 GB per replica group
    Reference (DP=1): 140 GB per replica
    Reward (DP=1):     26 GB
    Approximately: 64 GPUs for policy training, 32 for rollout, 16 for reward
    Remaining 16 GPUs: spare / for reference model inference

PPO TRAINING INSTABILITY MONITORING:
  Reward trend:          increasing monotonically is good; plateauing early is bad
  KL divergence:         should stay < 0.1; > 0.3 indicates policy collapse
  Policy entropy:        should remain above a threshold; collapse → mode seeking
  PPO clip fraction:     should be 0.1-0.3; > 0.5 means clipping too aggressively
  Advantage statistics:  normalize advantages; large variance = high reward noise
```

10.7.3 DPO — The Simpler Alternative

```
DIRECT PREFERENCE OPTIMIZATION (DPO) — INFRASTRUCTURE VIEW:
```

```
  DPO replaces PPO's rollout-reward-KL loop with offline preference optimization.
  Models: Policy + Reference (frozen SFT model) — NO reward model needed.

  Memory: 2× model memory (vs PPO's 3-4×)
  For 70B: 2 × 140 GB = 280 GB (fits on 4× H100 with ZeRO-3)

  Training loop:
    For each (prompt, chosen_response, rejected_response):
      policy_chosen_logp   = policy.log_probs(chosen)
      policy_rejected_logp = policy.log_probs(rejected)
      ref_chosen_logp      = reference.log_probs(chosen)   # no_grad
      ref_rejected_logp    = reference.log_probs(rejected) # no_grad

      dpo_loss = -log_sigmoid(β × (
          (policy_chosen - ref_chosen) - (policy_rejected - ref_rejected)
      ))

    Standard backward + AllReduce — identical infrastructure to pretraining.

  RECOMMENDATION HIERARCHY:
    Start with DPO: simpler infra, comparable alignment for instruction following.
    Move to PPO: needed for complex reasoning, math, code (RL signal required).
    Consider GRPO (Group Relative Policy Optimization):
      PPO variant without value model.
      Saves 1× model copy vs full PPO.
      Used in DeepSeek-R1 training.
```

10.8 Pre-Launch Checklist and Eight Numbers

10.8.1 Pre-Launch Checklist

```
70B+ TRAINING RUN PRE-LAUNCH CHECKLIST:

□ PARALLELISM
  □ TP ≤ 8 (within NVLink domain — never cross IB with TP)
  □ Bubble fraction < 10% (M ≥ 4 × PP verified)
  □ Memory per GPU calculated and verified with 10% headroom
  □ nccl-tests allreduce_perf confirms expected bandwidth

□ PRECISION
```

```
  ☐ BF16 autocast enabled (not FP16 — overflow risk on large models)
  ☐ FP32 optimizer state confirmed (not BF16 optimizer — stability risk)
  ☐ FP8 Transformer Engine enabled if H100+ (after initial convergence)
  ☐ Loss scaling disabled (not needed with BF16)

☐ COMMUNICATION
  ☐ NCCL_IB_HCA matches GPU-to-NIC affinity (rail-optimized topology)
  ☐ Gradient overlap verified: nsys trace shows NCCL overlapped with backward
  ☐ Bucket size tuned (25-50 MB for 70B+)
  ☐ NCCL_P2P_DISABLE=0 (NVLink P2P enabled)
  ☐ NCCL timeout set: NCCL_TIMEOUT_SECONDS=1800

☐ FAULT TOLERANCE
  ☐ Async checkpointing configured (not synchronous)
  ☐ Checkpoint interval set to sqrt(2 × T_ckpt × T_MTTR) seconds
  ☐ SHA-256 checksums on checkpoint shards (validate before resume)
  ☐ Hot-standby nodes allocated (≥ 5% of cluster)
  ☐ DCGM XID monitoring active on all nodes; alerts wired
  ☐ SLURM --requeue set in job script
  ☐ Gradient norm clip configured (max_norm=1.0); spike alerting enabled

☐ DATA PIPELINE
  ☐ Sequence packing enabled (for variable-length datasets)
  ☐ Async prefetch: num_workers ≥ 4
  ☐ pin_memory=True in DataLoader
  ☐ Shard size > 100M tokens per file
  ☐ Data pipeline profiling: verify GPU never idle waiting for batch

☐ MONITORING
  ☐ Loss curve, gradient norm, LR schedule logged per step
  ☐ MFU computed and logged (target > 40%; alert at < 30%)
  ☐ NCCL AllReduce timing logged (custom NVTX markers or Nsight)
  ☐ IB link speed monitoring: alert at < 350 Gbps (from 400 Gbps)
  ☐ Per-step timing P99 monitored; alert at > 2× P50
```

10.8.2 Eight Numbers Every Training Engineer Knows Cold

```
THE EIGHT NUMBERS:

  #1   16 bytes/param
       Full Adam training memory per parameter.
       4B weights + 2B BF16 grad + 4B FP32 master + 4B m + 4B v = 18B (→ round
16B)
```

```
#2   TP ≤ 8
     Maximum safe tensor parallel degree (one NVLink domain).
     Violate this: TP latency blows up 12× over InfiniBand.

#3   (PP-1) / (PP-1+M) = bubble fraction
     M must be >> PP to keep bubble below 10%.
     PP=4, M=32: bubble = 8.6%. PP=4, M=8: bubble = 27% (too high).

#4   sqrt(2 × T_ckpt × T_MTTR) = optimal checkpoint interval (seconds)
     T_ckpt = checkpoint write time; T_MTTR = recovery time.
     Derivation: minimize total overhead from checkpointing + recovery.

#5   3× AllReduce bandwidth
     ZeRO Stage 3 (FSDP FULL_SHARD) requires 3× the communication vs naive DP.
     Offset by: overlap with compute. Net overhead: 5-15% of step time.

#6   ~1%/GPU/week
     Conservative GPU hardware failure rate (all fault types combined).
     10K GPUs = job interruption every ~1.7 hours without fault tolerance.

#7   30% extra compute
     Activation recomputation overhead (naive recompute-all strategy).
     Min-cut (torch.compile): < 5% extra compute for same memory savings.

#8   15-25% throughput gain from sequence packing
     Free throughput: eliminate padding waste in variable-length datasets.
     Always enable for any training with documents < seq_len.
```

10.9 Training Run War Stories [r1]

10.9.1 What Real Failures Look Like at 10K-GPU Scale

```
FAILURE ANATOMY — A REAL 10K-GPU TRAINING INCIDENT:

Timeline:
──────────────────────────────────────────────────────────────────────

T+00:00  Job starts. 10,240 H100s, LLaMA-class 405B model.
         MFU stabilizes at 52% within 200 steps. All green.

T+18:00  On-call gets paged: "Step time P99 > 2× P50 for 15 minutes."
```

```
         Loss curve on W&B: normal. DCGM health dashboard: all green.
         First instinct: transient; wait and see.

T+18:45  Step time variance NOT resolved.
         P50 step: 2.1s (normal). P99 step: 5.8s (2.75×).
         MFU dropped from 52% to 38%.
         Nothing in DCGM ECC dashboard.

T+19:15  NCCL_DEBUG=INFO on one node → enormous log volume.
         AllReduce ring timings show: ONE ring completing in 0.8s (normal),
         ANOTHER ring taking 3.2 seconds.
         Location: inter-node DP AllReduce, ring 7 of 8.

T+19:45  ibstatus on all nodes: ONE node with IB link at 100 Gbps.
         Should be 400 Gbps. Link degraded, not failed.
         DCGM never alerted — link didn't fail; it slowed.
         Degraded link bottlenecked ring 7, stalling ALL barrier syncs
         because AllReduce is a blocking collective.

T+20:15  Decision: drain node, replace with hot standby.
         Duration: 22 minutes. Resume from checkpoint (6 steps before failure).
         Wasted compute: ~2.5 hours × 10,240 GPUs.
         Cost: ~$340,000 at market rates.

LESSONS FROM THIS INCIDENT:

  1. DCGM default monitoring does NOT catch degraded (partial) links.
     Add explicit IB link speed monitoring:
     Expected 400 Gbps; alert at < 350 Gbps.
     Use: ibstatus | grep "Physical state", or custom DCGM plugin.

  2. Ring AllReduce masks stragglers poorly.
     The slowest ring sets the barrier for ALL rings.
     One slow link = entire cluster slows proportionally to AllReduce's
     share of step time.
     Consider: tree AllReduce for heterogeneous IB environments.

  3. P99 step time monitoring is the most sensitive early warning signal.
     P50 can look normal for hours while P99 shows the problem.
     Set alert: P99 > 1.5× P50 for 5 consecutive steps.

  4. The first instinct ("transient; wait and see") cost 45 minutes.
```

```
    At 10K-GPU scale: a $340K incident is not a transient.
    Protocol: investigate within 15 minutes if P99 > 1.5× P50.
```

10.9.2 Loss Divergence Diagnosis Framework

```
LOSS DIVERGENCE INVESTIGATION PROTOCOL:

  DIVERGENCE SYMPTOMS:
    Sudden loss spike (NaN/inf): hardware fault or gradient overflow
    Gradual loss plateau:         data quality, learning rate, optimizer issue
    Loss spike then recovery:     stale checkpoint resume with wrong LR schedule
    Oscillating loss:             batch size / LR interaction

  DIAGNOSIS DECISION TREE:

  Step 1: When did it start?
    Exact step number from W&B. Correlate with:
      LR schedule transitions (warmup end, decay start)
      Checkpoint resumes (checkpoint corruption?)
      Data shard transitions (bad data batch?)
      Hardware events (DCGM log for that step range)

  Step 2: Hardware or software?
    Run one forward pass on CPU: same divergence? → SOFTWARE bug
    No divergence on CPU: → HARDWARE (ECC, memory corruption)
    Check DCGM for correctable ECC errors in that time window.

  Step 3: Data or model?
    Log per-sample loss for 1,000 batches around divergence onset.
    Look for: single outlier batch with 100× higher loss → data issue.
    Check data deduplication: exact duplicate sequences → memorization + loss
drop.

  Step 4: Gradient norm analysis:
    Normal gradient norm for 70B BF16: ~0.5-2.0
    Spike to > 10: something abnormal in that step's batch or model state.
    Sustained increase: learning rate too high or model growing unstable.

  GRADIENT NORM MONITORING CODE:
    grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    # Log grad_norm every step to W&B / MLflow
```

```
    # Alert: grad_norm > 5.0 for 3 consecutive steps → investigate
    # Alert: grad_norm = NaN → immediate job suspend + checkpoint + investigate
```

10.10 Resuming and Recovery — Checkpoint Validation [r1]

10.10.1 Checkpoint Validation Before Resume

```
NEVER RESUME WITHOUT CHECKPOINT VALIDATION.

An interrupted checkpoint write produces a silently corrupted file.
Resuming from it: waste the time from the previous valid checkpoint
until the corruption is discovered mid-run.

VALIDATION STEPS BEFORE RESUME:

Step 1: File integrity (metadata check)
  Verify all expected shard files exist.
  Check file sizes against expected sizes (written at checkpoint time).
  Verify SHA-256 checksums against the manifest written at checkpoint creation.
  Reject: any file where checksum mismatches.

Step 2: Load-time validation (tensor shapes)
  Load checkpoint on one GPU.
  Verify all tensor shapes match current model architecture.
  Verify optimizer state shapes match (optimizer_state['state'][key].shape).
  Reject: any shape mismatch (indicates truncated write or wrong model).

Step 3: Sanity forward pass
  Load model from checkpoint.
  Run one forward pass on a fixed reference batch.
  Compare loss against the logged loss at checkpoint step.
  Reject: if loss diverges by > 5% (indicates corrupted weights).

Step 4: Training state verification
  Verify step counter matches expected value.
  Verify learning rate matches schedule at that step.
  Verify RNG state (critical for reproducibility if needed).
```

```python
```

```python
# Production checkpoint validation implementation:
import torch, hashlib, json, os

def validate_checkpoint(checkpoint_dir: str, strict: bool = True) -> dict:
    """Validate training checkpoint before resuming."""
    results = {"valid": True, "errors": [], "warnings": []}

    # Step 1: File integrity
    manifest_path = os.path.join(checkpoint_dir, "manifest.json")
    if not os.path.exists(manifest_path):
        results["warnings"].append("No manifest.json found; skipping checksum valida
    else:
        manifest = json.load(open(manifest_path))
        for fname, expected_hash in manifest.get("checksums", {}).items():
            fpath = os.path.join(checkpoint_dir, fname)
            if not os.path.exists(fpath):
                results["errors"].append(f"Missing file: {fname}")
                results["valid"] = False
                continue
            actual_hash = hashlib.sha256(open(fpath, "rb").read()).hexdigest()
            if actual_hash != expected_hash:
                results["errors"].append(f"Checksum mismatch: {fname}")
                results["valid"] = False

    if not results["valid"] and strict:
        raise ValueError(f"Checkpoint validation failed: {results['errors']}")

    # Step 2: Load and verify shapes
    ckpt_path = os.path.join(checkpoint_dir, "model_state.pt")
    try:
        state = torch.load(ckpt_path, map_location="cpu")
        results["step"] = state.get("step", "unknown")
        results["loss_at_ckpt"] = state.get("loss", None)
    except Exception as e:
        results["errors"].append(f"Failed to load checkpoint: {e}")
        results["valid"] = False

    return results


def resume_with_validation(checkpoint_dir: str, model, optimizer, scheduler):
    """Resume training after checkpoint validation."""
    validation = validate_checkpoint(checkpoint_dir)
```

```python
    if not validation["valid"]:
        # Look for previous valid checkpoint
        prev_dir = find_previous_valid_checkpoint(checkpoint_dir)
        if prev_dir:
            print(f"Current checkpoint invalid; falling back to {prev_dir}")
            return resume_with_validation(prev_dir, model, optimizer, scheduler)
        else:
            raise RuntimeError("No valid checkpoint found; cannot resume.")

    # Load validated checkpoint
    state = torch.load(os.path.join(checkpoint_dir, "training_state.pt"))
    model.load_state_dict(state["model"])
    optimizer.load_state_dict(state["optimizer"])
    scheduler.load_state_dict(state["scheduler"])

    print(f"Resumed from step {validation['step']}, loss={validation['loss_at_ckpt']
    return state["step"]
```

10.11 Key Takeaways and Review Questions

```
EIGHT NUMBERS TO MEMORIZE COLD (from §10.8.2):
  16 bytes/param    — Full Adam training memory
  TP ≤ 8           — NVLink domain limit
  (PP-1)/(PP-1+M)  — Bubble fraction formula
  sqrt(2×Tc×Tr)    — Optimal checkpoint interval
  3× AllReduce     — ZeRO Stage 3 bandwidth cost
  ~1%/GPU/week     — GPU failure rate
  30% compute      — Naive activation recompute overhead
  15-25% gain      — Sequence packing throughput benefit

KEY PRINCIPLES:

  1. PARALLELISM HIERARCHY IS RIGID:
     TP within NVLink (≤8 GPUs), PP one IB hop, DP across cluster.
     Violating any of these is catastrophic, not marginal.
     Design parallelism strategy before choosing hardware, not after.

  2. MFU IS THE NORTH STAR METRIC:
     Profile six components before optimizing any one (compute, comm,
     memory, I/O, launch overhead, pipeline bubble).
```

```
   MFU < 40%: almost always a communication or pipeline problem.
   MFU 40-55%: healthy range for large-scale training.
   MFU > 55%: excellent — typically achieved only with perfect overlap.

3. FAULT TOLERANCE AT 10K GPUs = EVERY 1.7 HOURS:
   Async checkpointing is non-negotiable above 1,000 GPUs.
   Optimal checkpoint interval: sqrt(2 × T_ckpt × T_MTTR).
   Always validate checkpoint integrity before resume.
   Never dismiss P99 step time spikes — they predict imminent hardware failure.

4. SEQUENCE PACKING IS FREE THROUGHPUT:
   Always enable for variable-length text datasets.
   15-25% throughput gain from eliminating padding waste.
   Implementation: FlashAttention varlen mode + document attention masks.

5. RLHF IS A SYSTEMS PROBLEM, NOT JUST AN ML PROBLEM:
   PPO requires 4 concurrent subsystems: rollout, reward, reference, update.
   Memory: 3-4× a standard pretraining run.
   Start with DPO (2 models, standard training loop) before committing to PPO.
   GRPO (DeepSeek-R1 style) as middle ground: no value model needed.
```

Review Questions

Difficulty: Foundational

Question 1 (Memory Accounting): A new model has 13B parameters. You want to fine-tune it with AdamW (momentum + variance + FP32 master weights + BF16 compute weights + BF16 gradients). (a) Calculate the total training state memory. (b) How many H100 GPUs (80 GB each) are needed at minimum without any sharding? (c) With FSDP ZeRO Stage 3 across 8 GPUs, how much memory does each GPU need for model state (excluding activations)?

Expected answer: (a) 13B × (2B BF16 weights + 2B BF16 grad + 4B FP32 master + 4B Adam m + 4B Adam v) = 13B × 16B = 208 GB. (b) 208 GB / 80 GB = 2.6 → minimum 3 H100s just for training state (before activations). (c) ZeRO Stage 3 shards everything across DP replicas. With 8 GPUs: 208 GB / 8 = 26 GB per GPU for model state. Plus activations and framework overhead → ~40-50 GB per GPU, fits in 80 GB H100.

Difficulty: Intermediate

Question 2 (Bubble Fraction and Pipeline Schedule): You are training a 70B model with PP=8 pipeline stages. Currently using M=8 micro-batches per step. (a) Calculate the pipeline bubble fraction. (b) What is the minimum M to achieve < 10% bubble? (c) If you switch to interleaved

1F1B with V=2 chunks per stage, what M is needed for < 10% bubble? What is the memory tradeoff?

Expected answer: (a) bubble = (PP-1)/(PP-1+M) = 7/(7+8) = 7/15 = 46.7% — extremely high, wasting nearly half of compute time on bubbles. (b) For bubble < 0.10: (PP-1)/(PP-1+M) < 0.10 → 7/(7+M) < 0.10 → 7 < 0.10×(7+M) → 70 < 7+M → M > 63. Need M ⩾ 64 for < 10% bubble with PP=8 non-interleaved. (c) Interleaved 1F1B with V=2: bubble = (PP-1)/(PP-1+M×V) = 7/(7+M×2) < 0.10 → M > 28. Need M ⩾ 32 for < 10% bubble. Memory tradeoff: V=2 chunks means 2× more in-flight activations compared to standard 1F1B. Activation memory per stage doubles. If activations were the memory bottleneck, this may require reducing micro-batch size — potentially undoing the bubble improvement.

Difficulty: Principal Level

Question 3 (Full Training Architecture for 175B): Design the complete 4D parallelism configuration for training a 175B parameter dense transformer (96 layers, d_model=12,288) on 2,048× H100 SXM5 (256 nodes × 8 GPUs/node). Show memory math proving the config fits in 80 GB HBM. Calculate the pipeline bubble fraction and determine whether interleaved scheduling is warranted. What changes if sequence length increases from 4K to 128K tokens?

Expected answer: Memory estimate for 175B: 175B × 16B = 2,800 GB training state. TP=8 (within each 8-GPU node): each TP group holds 1/8 of weights = 2,800/8 = 350 GB per TP group. Still doesn't fit on 8 × 80 GB = 640 GB without ZeRO. PP=4 (split 96 layers into 4 stages of 24 layers each): weight per stage = 350/4 = 87.5 GB per GPU — still too large. Add ZeRO-2 (gradient + optimizer sharding) within DP group: with DP = 2048/(8×4) = 64 DP replicas, ZeRO-2 reduces gradient+optimizer memory by ~1/DP factor. Optimizer state: 175B × 12B / (8 TP × 4 PP × 64 DP) = 2,100 GB / 2,048 = ~1 GB per GPU. Weight + compute: 175B × 2B / (8 × 4) = 10.9 GB per GPU. Total model state: ~12 GB per GPU — fits well within 80 GB. Activation memory (S=4K, per layer, with checkpointing): 4096 × 12,288 × 2B = 100 MB. 24 layers per stage: 2.4 GB per GPU — no problem. Final config: TP=8, PP=4, DP=64, ZeRO-2. Total GPUs: 8×4×64 = 2,048 ✓. Bubble fraction with M=64 microbatches: (4-1)/(3+64) = 3/67 = 4.5% — good. No interleaved schedule needed. At S=128K: activation memory per layer = 128,000 × 12,288 × 2B = 3.15 GB. 24 layers × 3.15 GB = 75.5 GB per GPU — nearly fills all 80 GB with just activations! Must add CP. With CP=16: each GPU holds 128K/16 = 8K tokens locally for attention, reducing activation memory 16×: 75.5/16 = 4.7 GB per GPU. New config: TP=8, PP=4, DP=8, CP=16, ZeRO-3. DP reduces to 2,048/(8×4×16) = 4 replicas — very low data parallelism. Alternative: reduce PP to 2, increase DP to 16. Either way, 128K context drastically changes the parallelism architecture.

Difficulty: Principal Level

Question 4 (Failure Mode Diagnosis): An Nsight Systems profile of a 70B training run on 512× H100s shows:

- Forward pass: 450ms
- Backward pass: 900ms
- NCCL AllGather + ReduceScatter (FSDP): 480ms (appears *after* backward in the trace)
- Optimizer step: 80ms
- Total step time: 1,910ms (a) Calculate current MFU assuming H100 peak BF16 = 989 TFLOPS. (b) If perfect overlap achieved, what is theoretical minimum step time? (c) List exact FSDP configuration parameters to enable overlap. (d) Explain why NCCL overlap might fail even with correct FSDP config.

Expected answer: (a) FLOPs per step: 6 × 70B × (micro_batch × S) tokens. Assume micro_batch=4, S=4096: 6 × 70B × 4 × 4096 = 6.87 × 10^15 FLOPs. Compute per GPU: 6.87e15 / 512 = 1.34e13 FLOPs. At 1,910ms: actual TFLOPS = 1.34e13 / 1.91 = 7.02 TFLOPS per GPU. MFU = 7.02 / 989 = 0.71%. That's extremely low — reconsider assumption. More likely global batch = 512 × 4 × 4096 tokens = 8.6B tokens/step. FLOPs = 6 × 70B × 8.6B = 3.62e21? No — FLOPs per step = 6 × N_params × D_tokens_per_step = 6 × 70B × (512 GPUs × 4 × 4096 tokens) = 6 × 70B × 8.4M = 3.53e18. Per GPU: 3.53e18/512 = 6.9e15. At 1.91s: 3.61 TFLOPS/GPU. MFU = 3.61/989 = 0.37% — still very low. The calculation reveals 1,910ms per step is itself the problem indicator. Let's assume: with proper overlap, step ≈ max(forward+backward, NCCL) ≈ max(1,350, 480) = 1,350ms. MFU at 1,350ms: MFU = (6.9e15/1.35) / (989e12) = 5.1e15/9.89e14 = 5.2× — that's > 100% which means my token estimate is off. The key answer: current MFU = 1,350/1,910 of what it could be with overlap = 71% of potential. The NCCL appearing after backward is the problem. (b) Theoretical minimum: max(forward+backward, NCCL) = max(1,350ms, 480ms) = 1,350ms. Adding optimizer: 1,430ms. Improvement: 1,910 → 1,430 = 25% step time reduction. (c) FSDP overlap parameters: `BackwardPrefetch.BACKWARD_PRE` (prefetch next layer's params while computing current layer's backward), `forward_prefetch=True` (AllGather params for next layer while executing current), `limit_all_gathers=True` (memory safety), `use_orig_params=True` (required for torch.compile integration which enables overlap tracking). Also: set bucket sizes via `optim_state_dict_config`. (d) NCCL overlap fails even with correct FSDP config when: (1) Gradient norm clipping before AllReduce: `clip_grad_norm_` requires all gradients, blocking NCCL launch until full backward completes. Fix: clip after AllReduce, or compute norm via separate AllReduce call. (2) Custom backward hooks executing synchronous Python code (print statements, loss accumulation) that block the CUDA stream before NCCL can launch. Fix: make hooks async or move to end of step. (3) CUDA stream priority misconfiguration: NCCL stream has lower priority than compute stream, causing the GPU scheduler to deprioritize communication. Fix: ensure NCCL stream is CUDA priority 0 (default).

Further Reading

1. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism* — Shoeybi et al. (2019). `arxiv.org/abs/1909.08053` The foundational TP paper. Read Section 3 (model parallelism) and Section 4 (pipeline).
2. *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM* — Narayanan et al. (2021). `arxiv.org/abs/2104.04473` The 4D parallelism paper. The memory model and bubble fraction derivations are exact.
3. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models* — Rajbhandari et al. (2020). `arxiv.org/abs/1910.02054` The ZeRO paper. Read Section 3 (ZeRO stages 1-3) and the memory formula derivations.
4. *The Llama 3 Herd of Models* — Meta AI (2024). `arxiv.org/abs/2407.21783` Section 3.3 describes the 4D parallelism configuration for LLaMA-3 405B training. The description of the 16K GPU cluster and parallelism choices is invaluable.
5. *PyTorch Fully Sharded Data Parallel Documentation*. `pytorch.org/docs/stable/fsdp.html` The API reference. Read the `BackwardPrefetch` and `ShardingStrategy` sections.
6. *NCCL Documentation — Collective Operations*. `docs.nvidia.com/deeplearning/nccl/user-guide/` Ring AllReduce algorithm and the communication bandwidth formulas.

— End of Chapter 10 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved